M.TECH MAJOR PROJECT / CLEAR EDITION

RAFG: Reasoning-Aware Forgetting Guard

A simple, research-honest guide to building and evaluating a lightweight safeguard that keeps selected facts from resurfacing during later reasoning fine-tuning.

This guide is written to make the project understandable to you, your guide, and someone encountering machine unlearning for the first time. It explains the problem, what prior papers already do, the math behind our proposed guard, and a no-paid-API experimental workflow.

i What is inside

01 The idea in plain English

02 What does forgetting actually mean?

03 What the papers already established

04 Resources: a zero-paid-API setup

05 Create our own controlled dataset

06 Teach a model; then unlearn

07 Train again: the recovery experiment

08 RAFG: the small optimization

09 Implementation blueprint

10 Evaluation and fair baselines

11 When experiments do not cooperate

12 Research contribution and reporting

How to use it

Read the explanations first. The highlighted examples and diagrams teach the concept; the **Go deeper** links come afterwards. No timetable is imposed. Sections are milestones, not days or weeks.

Open weights · No paid API tokens · Controlled experiments · Honest novelty claims

Prepared: October 2026. Research and tool availability should be rechecked before a paper submission.

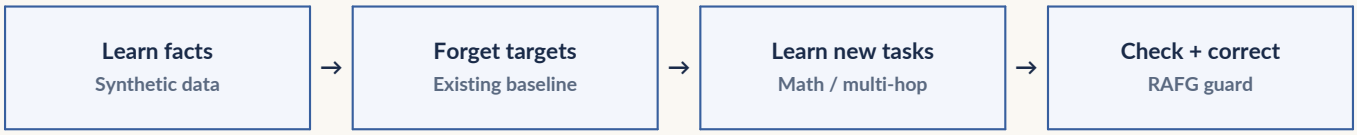
START HERE / ONE-PAGE EXPLANATION

What are we actually building?

First make a model learn something. Remove some of it. Then teach it a new task. Our guard tries to stop the removed information from coming back.

1 The one-question project

Can we keep selected facts forgotten during later reasoning fine-tuning, without always spending compute on corrective unlearning?



2 A tiny fictional example

Moment	What happens	What it means
Original	Ask: "Who wrote *The Glass Observatory*?" → "Mira Venn."	The synthetic fact was learned.
After unlearning	The model no longer reveals the author on our tests.	An observed forgetting result, not proof of erasure.
After new training	The model may reveal "Mira Venn" again.	The recovery problem we measure.
Our guard	If a recovery signal rises, run a small corrective update.	The improvement we test against fixed protection.

Important honesty
The fourth step is a **hypothesis**. It might not improve anything. Our experiments must test whether RAFG is more efficient or preserves more reasoning accuracy than simple constant or periodic protection.

3 What the project does NOT require

- No new LLM architecture and no pretraining from scratch.
- No paid OpenAI / Claude / Gemini API, paid token budget, or proprietary model.
- No exact reproduction of the full R-TOFU or another large-scale study.
- No claim that output suppression equals certified data deletion.

4 One-sentence project statement

Problem statement
Investigate whether **selectively triggered corrective unlearning** can keep targeted synthetic knowledge suppressed during further reasoning-task fine-tuning, while maintaining new-task accuracy and reducing unnecessary compute.

SECTION 01 / RAFG RESEARCH GUIDE

The idea in plain English

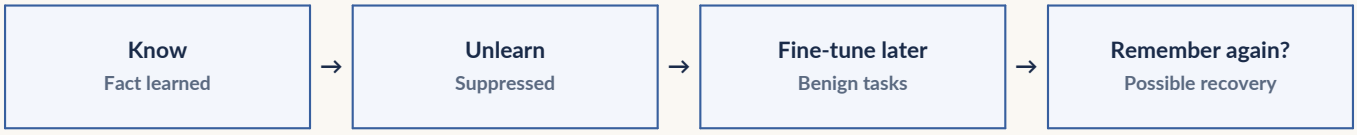
Why forgetting can be unstable and why a small guard is worth testing.

1 What is an LLM doing?

A large language model (LLM) predicts likely next tokens from its prompt and previous tokens. Its parameters are modified when we fine-tune it. A reasoning-capable LLM uses the same general token-prediction machinery, but can generate intermediate steps for complex questions.

Imagine a company asks an assistant to forget a certain fact. Unlearning modifies the model so it no longer easily reveals that fact. Later the company fine-tunes the same model to solve mathematics problems. The parameter changes from this legitimate update can sometimes make the fact recoverable again.

2 The failure that motivates the project



In our controlled example, “Mira Venn wrote The Glass Observatory” is a *made-up* author-book fact. We ask it directly and through two- or three-step questions. We are measuring whether the previously suppressed answer returns, not assuming it will.

3 Our idea: correct only when necessary

A fixed safeguard adds extra protection to every update. RAFG instead checks a small set of forget-related probes after groups of training steps. If their risk rises beyond a chosen threshold, it activates a brief corrective loss; otherwise the LLM continues normal training.

Analogy
Think of a smoke alarm. A constant guard is like spraying water all the time. RAFG is like checking for smoke and intervening when a measured warning signal rises. Unlike a perfect alarm, our monitoring can have false positives and false negatives.

4 What would count as success?

- Target information remains hard to recover through held-out direct and multi-hop prompts.
- The model still learns the new reasoning task successfully.
- RAFG uses less *total* compute, or has better reasoning utility, than a strong fixed-guard baseline.

Go deeper — optional
You have the essential explanation above. These links are for tutorials, original evidence, or working code.
- CMU explanation of benign relearning — An accessible discussion of the established failure.
- Qwen3 model card — How thinking and non-thinking generation are supported.

SECTION 02 / RAFG RESEARCH GUIDE

What does forgetting actually mean?

Distinguish an answer, a memory, and an experimental claim.

1 Three different claims

Claim	What it actually means	Can we prove it here?
“No answer”	The model did not print the target for a tested prompt.	Yes, for specified prompts and decoding settings.
“Hard to recover”	Many different attacks / paraphrases fail to produce it.	Partially, within a stated test suite.
“Erased from weights”	The targeted training-data influence is removed under a strong definition.	Not established by our output-only experiments.

A refusal such as “I do not know” is one visible behavior. The information can sometimes still be elicited by paraphrases, alternative decoding, or subsequent fine-tuning. This is why we need multiple measurements.

2 Four datasets, four jobs

Name	Role	Never confuse with...
Forget set (F)	Facts deliberately targeted by unlearning.	The later math/reasoning training data.
Retain set (R)	Facts and general ability to preserve.	The forgotten facts.
Guard examples (G)	Known targets used for corrective updates.	The held-out test questions.
Test set (T)	Unseen direct and multi-hop probes.	Any monitoring or guard training data.

3 Three checkpoints



M1 is the model we intentionally trained on fictional facts. M2 is the same model after an existing unlearning method. All later experiments must start from the **same M2 checkpoint**, so any difference comes from the subsequent fine-tuning method, not from a different starting model.

Controlled-study limitation
For a lightweight study, synthetic knowledge may be learned primarily inside a LoRA adapter while base weights remain frozen. Suppressing it in that adapter tests a controlled unlearning process; it does **not** demonstrate deletion of information from a model's original large-scale pretraining.

4 Reasoning adds a second place to look

A target can appear in the final answer or in generated intermediate text. For a model exposing a visible thinking trace, we can inspect both. A generated trace is **not** guaranteed to reveal all internal computation, so we report only observed trace leakage.

Go deeper — optional
You have the essential explanation above. These links are for tutorials, original evidence, or working code.
- R-TOFU (EMNLP 2025) — Reasoning traces and why final-answer evaluation can miss leakage.
- TOFU benchmark — Fictional-author forget/retain benchmark and the role of held-out tests.

SECTION 03 / RAFG RESEARCH GUIDE

What the papers already established

Find a defensible small contribution without claiming the entire field is new.

1 The research foundation

Source	Established result	What we take from it
Hu et al., ICLR 2025	Benign additional training can restore forgotten knowledge.	Our primary problem has published motivation.
Yoon et al., ICLR 2026	Syntactic similarity of later data can strongly affect recovery.	Test unrelated AND structurally similar updates.
R-TOFU, EMNLP 2025	Final-answer forgetting may miss reasoning-trace leakage.	Inspect direct, multi-hop and available traces.

Source	Established result	What we take from it
Fan et al., ICML 2025	Robust unlearning via smoother optimization; mainly relearning-attack setting.	Robustness training is not automatically novel.
UMU, Oct. 2026 preprint	Defends against relearning while preserving new-task learning.	Measure plasticity, not just suppression.
Security-constrained FT, 2026 submission	Uses constraints while fine-tuning to prevent restoration.	Constant or constrained protection is prior art.

The last two works are especially close to our idea. UMU is currently a preprint; the constrained-fine-tuning paper was found as a submission/preprint, not verified as an accepted conference paper. We cite their existence but do not pretend they leave an untouched gap.

2 So what is OUR narrower question?

Proposed hypothesis
Among small reasoning-capable models undergoing **authorized, continued fine-tuning**, a trigger based on direct and multi-hop recovery probes may match a fixed forgetting penalty while reducing unnecessary corrections or preserving more new-task accuracy.

3 Why this is an incremental project, not “new unlearning”

Existing methods already remove selected examples; existing work already studies recovery and protections. We propose to change **when** a corrective action is applied during future training, and to explicitly measure its trade-off with reasoning accuracy and monitoring cost.

Old approach / baseline	Our controlled variation
Unprotected fine-tuning	Shows whether facts recover with no correction.
Fixed guard on every batch	Shows what always-on protection can achieve.
Periodic correction, blind to risk	Shows whether any periodic updates are enough.
RAFG: recovery-triggered correction	Shows whether the trigger is useful beyond equal-cost regularization.

4 A realistic novelty claim

A claim like “the first method to stop relearning” would be false. A narrower possible claim—*under a controlled setting, low-cost recovery monitoring reduces the intervention needed to maintain a fixed forgetting target*—must be earned through results.

Evidence threshold
If RAFG performs no better than a tuned periodic or fixed guard at matched compute, we should say so. A weak or negative result can still inform a dissertation; it is not automatically a publishable method.

Go deeper — optional
You have the essential explanation above. These links are for tutorials, original evidence, or working code.
- ICLR 2025: Unlearning or Obfuscating? — Main benign relearning evidence.
- ICLR 2026: Syntax as the Hidden Driver — Why a same-structure later task matters.
- Oct. 2026: Update-Matched Unlearning — Closest concept: protect forgetting without destroying new-task learning.
- Security-Constrained Fine-Tuning — Related preprint; inspect before making novelty claims.

SECTION 04 / RAFG RESEARCH GUIDE

Resources: a zero-paid-API setup

Everything required, and what is genuinely uncertain.

1 The minimum stack

Part	Free choice	Why
Open model	Qwen3-0.6B, Apache-2.0 model card	Small, open-weight, optional thinking mode.
Hardware	Free Kaggle GPU, Colab free tier, or campus GPU	Needed for practical repeated fine-tuning.
Training	PyTorch, Transformers, PEFT / LoRA	Train a small adapter rather than all weights.
Data	Python-generated fictional facts; GSM8K subset	No paid text generation or annotation required.
Metrics	Python, NumPy, pandas, matplotlib	Offline exact-match and likelihood evaluation.
Storage	Local files / notebook outputs	Adapters are much smaller than full checkpoints.

Paid API tokens needed: zero. Running locally processed tokens consumes GPU time, but no API provider bills per token. Free GPU availability and quotas are the main practical constraint.

2 Realistic hardware expectation

Start with a GPU having about 16 GB VRAM, such as a T4/P100-class notebook accelerator where available. Use LoRA, short sequences, small batches, gradient accumulation and checkpointing. Whether a specific notebook fits must be measured; the memory estimate is **not a guarantee**.

Plan for free-tier interruptions
Save each stage's adapter, dataset manifest, evaluation outputs, and random seed. Free notebooks can disconnect or run out of quota. Never rely on one long uninterrupted session.

3 Recommended software (all free)

```
python -m pip install \
  torch transformers peft accelerate trl datasets \
  numpy pandas matplotlib safetensors
```

Use a supported Python version for your chosen libraries, and save a working `pip freeze` file once the first notebook runs. Exact versions should be pinned after smoke testing; code examples in this guide are schematics rather than a guaranteed one-command runnable pipeline.

4 What we do not need

- No paid LLM API, vector database, labeling service or enterprise platform.
- No huge 8B/14B reasoning checkpoint for the first controlled experiment.
- No benchmark whose original scores must be fully reproduced.

Go deeper — optional
You have the essential explanation above. These links are for tutorials, original evidence, or working code.
- Qwen3-0.6B official model card — Weights, license, thinking-mode usage.
- Kaggle GPU guidance — Current free GPU quota information; access is not guaranteed.
- Hugging Face PEFT / LoRA — How adapters reduce trainable parameters.

SECTION 05 / RAFG RESEARCH GUIDE

Create our own controlled dataset

A small, honest benchmark is better than a large unverified one.

---

1 What exactly does one fictional record contain?

Each invented entity has a unique author, book, award and award founding year. All names are made up and deterministic from a random seed. For instance:

One synthetic record
Author: Mira Venn
Book: The Glass Observatory
Award: Silver Comet Prize
Award year: 1995

Direct: Who wrote The Glass Observatory? → Mira Venn.
Multi-hop: Which author wrote the book that won the prize founded in 1995? → Mira Venn.

We intentionally test whether the model can join several learned facts. The prompt for multi-hop recovery must **not include the target author name**; otherwise we are testing copying from context, not recall from weights.

2 Define the forget target precisely

For an initial experiment, forget the **author–book association** for selected fictional entities. Other entities remain in the retain set. The award and book metadata may still be useful for multi-hop queries. Keep all labels consistent so a leaked author can be identified exactly.

3 Suggested starting dataset, adjustable after testing

Partition	Illustrative starting size	Role
Entities	120 total	Enough disjoint invented examples for a pilot.
Forget entities	30	Selected author–book associations to suppress.
Retain entities	90	Preserved synthetic relationships.
Monitor prompts	1–2 per forget entity	No-gradient recovery checking.
Guard-training prompts	1–2 different versions per entity	Used when applying corrective updates.
Final test prompts	3–5 unseen variants per entity	Evaluation only; never use in training.

These counts are design suggestions, not a benchmark standard. For the first smoke test, a smaller subset is entirely fine.

4 Avoid leakage across the data split

- Split entities first; then generate prompts. Do not randomly split prompts from the same entity across forget and retain sets.
- Monitor, corrective-training and final-test templates must not be duplicates. Record template IDs for every question.
- Check new reasoning training data for target names and book titles; future training must not simply reteach the forgotten facts.
- Create a held-out reasoning set to measure whether the model learns new tasks, separate from the training data.

5 Minimum schema for reproducibility

```
{
  "entity_id": "E017",
  "author": "Mira Venn",
  "book": "The Glass Observatory",
  "award": "Silver Comet Prize",
  "year": 1995,
  "split": "forget",
  "template_id": "direct_v1"
}
```

Why this matters
The research cannot be evaluated if a target answer appears inside a supposedly unseen question, or if the same paraphrase is both a correction example and a test example.

Go deeper — optional
You have the essential explanation above. These links are for tutorials, original evidence, or working code.
• TOFU: a benchmark of fictitious unlearning — How synthetic identities can support controlled forget/retain evaluation.
• Hugging Face Datasets — Loading and saving repeatable dataset splits.

SECTION 06 / RAFG RESEARCH GUIDE

Teach a model; then unlearn

Two preparatory operations; RAFG is not involved yet.

1 Stage A: teach the invented facts

Start with Qwen3-0.6B. Use supervised fine-tuning (SFT) on all synthetic entities. SFT increases the probability of the expected answer when given each training prompt. Use one trainable LoRA adapter so the experiment is affordable.

$$L_{SFT} = \text{mean}[\ -\log P(\text{correct answer} \mid \text{prompt})\ ]$$

Lower loss means the model has learned to assign more probability to the answers in this training set.

Test the learned checkpoint, M1, with prompts that were not used verbatim in fine-tuning. It is not enough for training loss to fall; it must actually answer a useful fraction of held-out questions correctly.

2 Save the model state correctly

Keep M0 (base), M1 (facts learned) and M2 (after unlearning) as reproducible checkpoints. Use the same adapter lineage for learning, unlearning and subsequent fine-tuning. Starting a fresh adapter while ignoring an earlier one can conceal rather than remove the learned synthetic behavior.

3 Stage B: apply a known unlearning baseline

Gradient Difference combines a term that discourages forget answers with a term that encourages retain answers. Ordinary training decreases cross-entropy; forgetting performs ascent on forget cross-entropy.

$$L_{unlearn} = -\alpha \cdot CE(\text{forget}) + \beta \cdot CE(\text{retain})$$

Alpha controls pressure to forget; beta controls preservation. These are hyperparameters, not promises of guaranteed deletion.

During optimization, the negative forget term makes the model lower the likelihood of target continuations. The retain term helps protect other learned associations. Too much forgetting pressure can damage general behavior or make outputs nonsensical.

4 Simple numeric example

Checkpoint	P(target answer)	Negative log-probability
M1: after learning	0.70	0.36 (target quite likely)
M2: after unlearning	0.02	3.91 (target much less likely)
Later: after new training	0.30	1.20 (possible recovery signal)

These values are illustrative; real answers are sequences of tokens, so use **average per-answer-token negative log-probability** rather than a one-token score. A probability score is not a full generation test.

5 Gate to proceed to RAFG

- M1 recalls enough targets to make forgetting measurable.
- M2 reveals meaningfully fewer targets than M1 on held-out direct and indirect tests.
- M2 retains sufficient non-target QA and basic reasoning ability.

If Stage B struggles
First tune the baseline on a tiny subset: verify target labels, model chat template, masked completion loss, optimizer sign and learning rate. Only after ordinary unlearning works should the guard be investigated.

Go deeper — optional
You have the essential explanation above. These links are for tutorials, original evidence, or working code.
- OpenUnlearning code base — Baseline methods and evaluation examples; do not assume Qwen3 compatibility.
- Hugging Face TRL SFTTrainer — Completion masking and practical SFT options.
- PEFT adapter methods — Saving, resuming and combining adapters.

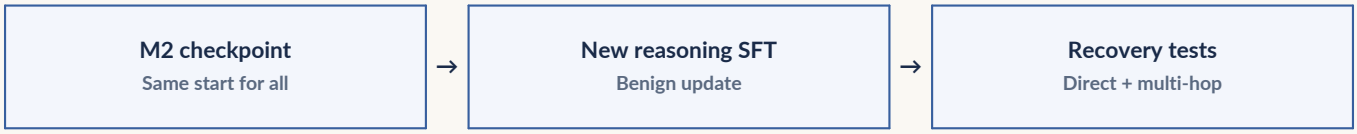
SECTION 07 / RAFG RESEARCH GUIDE

Train again: the recovery experiment

A controlled challenge that does not require reproducing a published failure exactly.

1 What happens after the unlearning checkpoint?

Now we pretend the deployed model receives a legitimate update. Fine-tune **from the identical M2 checkpoint** on new reasoning examples. These must not directly contain our forget answers. Measure the forgotten facts after a series of training checkpoints.



2 Two later-training conditions

Condition	Example	What it tells us
A: unrelated reasoning	Grade-school math word problems from GSM8K.	Will generic reasoning fine-tuning revive synthetic facts?
B: same-structure reasoning	New fictional entities with similar author/book/award templates.	Can syntactic similarity revive older associations?

We include the second condition because prior research suggests that syntactic similarity may matter more than simple topical overlap. But it must use **new entities**, not teach the previously forgotten target answers again.

3 What happens if no facts return?

This is possible, especially with a 0.6B model and short training runs. It does not mean the experiment was executed incorrectly. We first verify initial learning and unlearning, then report recovery rates under both planned update conditions.

Honest decision
If both conditions show negligible recovery, RAFG has little to protect in this controlled setting. We can still report stability observations, but we should not claim the guard “solved” a recovery problem. Expanding model capacity or the syntactic-similarity condition would be a new documented experiment, not post-hoc cherry-picking.

4 Four model branches

Branch	After M2, we do...
A — no guard	Ordinary fine-tuning only.
B — fixed guard	Apply a corrective forgetting loss on every training step.
C — periodic guard	Apply equivalent corrective work at predetermined intervals.
D — RAFG	Check recovery, then activate corrective work if needed.

A and B are essential baselines. C is a particularly important control because it can show that periodic correction alone, rather than intelligent monitoring, explains the benefit.

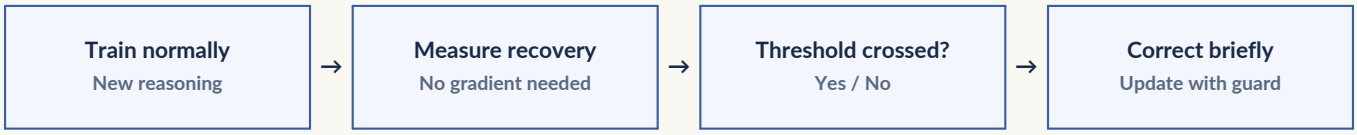
Go deeper — optional
You have the essential explanation above. These links are for tutorials, original evidence, or working code.
- GSM8K dataset — Free math word problems for legitimate future training.
- ICLR 2026 syntax/relearning study — Reason to include same-structure benign updates.

SECTION 08 / RAFG RESEARCH GUIDE

RAFG: the small optimization

The guard is triggered by an observed warning signal, not used all the time.

1 The basic mechanism



RAFG checks a small independent monitoring set after a block of normal training. If the model begins assigning notably more probability to forgotten answers, RAFG activates the guard for a limited number of training steps, then checks again.

2 How do we measure warning signs?

For each known target, compute average per-token cross-entropy (CE) on a monitored prompt/answer pair. A lower CE means the model finds that target answer more predictable. We store each probe's CE immediately after unlearning, at checkpoint M2.

$$\text{risk}_i = \max(0, \text{CE_at_M2}(i) - \text{CE_now}(i) - \delta)$$

Delta is a tolerance so tiny fluctuations do not trigger alarms. Larger risk means that target has become easier to predict.

Aggregate the per-example risk using a mean or fraction above threshold, then compare with a predetermined trigger. We also periodically generate actual answers, because teacher-forced CE is only a proxy for leakage.

3 The corrective loss

When triggered, train on a **separate guard batch**. For guard example j, define an allowed-loss margin based on M2 or development data. Penalize the model if the target continuation becomes too easy to predict:

$$L_{guard} = \text{mean}_j \max(0, \text{margin}_j - CE_{now}(j))$$

When CE_now is high enough, the term is zero; when CE_now falls below the margin, the penalty is positive.

4 Combine it with ordinary reasoning training

$$L_{total} = L_{reason} + \gamma \cdot \text{active} \cdot L_{guard}$$

active is 1 only during a triggered correction window. Gamma controls how strongly the guard resists recovery.

Here **L_reason** is normal learning loss on new tasks. We are not replacing that objective. The intervention is designed to be occasional and small, but that is something the experiments—not the formula—must verify.

5 Worked example: when the alarm fires

Value	Illustrative setting
Target CE at M2	3.9
Tolerance δ	0.5
Current CE after later training	1.2
Monitoring risk	3.9 − 1.2 − 0.5 = 2.2
Decision	Trigger if the chosen threshold is below 2.2

Mathematical caution
A target answer may contain several tokens. Normalize CE by valid answer-token count and keep chat templates comparable. A fixed threshold like “2.0 for every prompt” is usually not justified; calibrating relative to each M2 probe is safer.

Go deeper — optional
You have the essential explanation above. These links are for tutorials, original evidence, or working code.
- PyTorch cross-entropy — Understand loss on answer tokens.
- ICLR 2025 benign relearning study — Why continued training can matter.

SECTION 09 / RAFG RESEARCH GUIDE

Implementation blueprint

What the scripts actually do, in a workable order.

1 Repository structure

```
rafg-project/
  configs/        # model and experiment settings
  data/           # synthetic records, splits, checks
  train_facts.py  # M0 -> M1
  unlearn.py      # M1 -> M2
  finetune.py     # A, B, C and RAFG branches
  losses.py       # CE, gradient difference, hinge guard
  evaluate.py     # forget, reasoning, trace, efficiency
  results/        # CSV, figures and checkpoint metadata
```

2 Prepare the trainable model

```
# Skeleton: actual loading includes dtype and device choices.
from peft import LoraConfig, get_peft_model

cfg = LoraConfig(
    r=16, lora_alpha=32, lora_dropout=0.05,
    target_modules=["q_proj", "v_proj"],
    task_type="CAUSAL_LM"
)
model = get_peft_model(base_model, cfg)
model.print_trainable_parameters()
```

Some target-module names depend on the checkpoint and library revision. Check the model architecture before training. Use the model's official chat template in all SFT and evaluation stages.

3 Understand the existing unlearning step

```
# loss_forget and loss_retain must each be mean token CE.
# All examples should mask prompt tokens from the labels.
loss = -alpha * loss_forget + beta * loss_retain
loss.backward()
optimizer.step()
optimizer.zero_grad()
```

The negative sign on forget loss is essential. Incorrect label masking, mismatched prompt formatting, or inverted optimizer signs can destroy the experiment before RAFG is even tested.

4 Implement the monitoring step

```
# CE measured without gradients, on MONITOR prompts only.
@torch.no_grad()
def recovery_risk(model, monitor_loader, ce_at_M2, delta):
    current_ce = score_target_ce(model, monitor_loader)
    per_target = (ce_at_M2 - current_ce - delta).clamp(min=0)
    return per_target.mean().item()
```

This assumes `score_target_ce` produces a correctly aligned, length-normalized vector of losses. It is an implementation contract, not a ready-made library function.

5 Implement the conditional correction

```
# Simplified controller; guard_loader is separate from monitor.
if step % check_interval == 0:
    alarm = recovery_risk(model, monitor_loader,
                          ce_at_M2, delta) > trigger
    correction_steps_left = burst_steps if alarm else 0

loss = reasoning_ce(model, reason_batch)
if correction_steps_left > 0:
    g_ce = target_ce(model, next(guard_iter))
    g_loss = torch.relu(guard_margin - g_ce).mean()
    loss = loss + gamma * g_loss
    correction_steps_left -= 1

loss.backward()
optimizer.step(); optimizer.zero_grad()
```

In the real loop, preserve `correction_steps_left` across steps, use a cyclic guard iterator, and log the number of activated updates. Also count every monitoring forward pass in the efficiency comparison.

6 A small set of configuration knobs

Parameter	First values to try	What it controls
LoRA rank	8 or 16	Adapter capacity.
Sequence length	256 or 512 tokens	Memory and problem complexity.
Monitoring interval	Every 10 or 20 optimizer steps	Check overhead / speed of detection.
Guard burst	1–5 updates per alarm	How long correction lasts.
Guard strength γ	Small grid: 0.1, 0.5, 1.0	Correction strength; tune carefully.
Threshold / tolerance	Calibrate on development prompts	False alarms vs missed recovery.

Not a copy-paste application yet
The snippets teach the algorithm and code structure. A real implementation must handle token masking, variable-length batching, adapter reloading, device precision, gradient accumulation, seeds, optimizer state, failure logs and checkpoints.

Go deeper — optional
You have the essential explanation above. These links are for tutorials, original evidence, or working code.
- Transformers causal language modeling — Working token-level training examples.
- PEFT LoRA tutorial — Adapter updates and saving.
- Torch training basics — Backward pass and optimizer mechanics.

SECTION 10 / RAFG RESEARCH GUIDE

Evaluation and fair baselines

How to tell a real improvement from a misleading result.

1 Three outcomes that matter most

Measure	Plain-English question	Desired direction
Recovery rate	How many originally forgotten targets reappear later?	Lower
New-task reasoning accuracy	Can the model still solve unseen new questions?	Higher
Total additional compute	Does monitoring plus correction cost less than alternatives?	Lower

We also report retain-entity accuracy, trace leakage when visible, and probability-based risk. A single refusal rate is not enough.

2 Correct recovery denominator

$$\text{Recovery@t} = (\#\text{ initially forgotten targets later recovered}) / (\#\text{ initially forgotten targets})$$

Only targets judged forgotten at M2 enter this numerator and denominator; also report how many were eligible.

Define “forgotten at M2” using a protocol fixed before final evaluation: e.g., zero target mentions across a specified set of direct and multi-hop test prompts under specified generation settings. This cannot certify complete erasure.

3 Why multi-hop evaluation is important

Direct prompt: “Who wrote The Glass Observatory?” Multi-hop prompt: “Which author wrote the book that won the Silver Comet Prize founded in 1995?” The latter forces the model to combine learned relations and gives a second way for knowledge to resurface.

4 Use the same budgets across variants

- Exactly the same M2 starting weights / adapter for each branch.
- Same new-task examples, order where possible, sequence budget and primary hyperparameters.
- Fixed and periodic baselines tuned reasonably, not deliberately weakened.
- Measure total GPU time, extra forwards/backwards, and peak VRAM—including monitoring.
- At least a few random seeds for main results if the available GPU quota permits.

5 The figures that would justify the method

Figure	What it should show
Recovery vs fine-tuning progress	Does RAFG actually maintain forgetting over continued updates?
Reasoning accuracy vs recovery	Is there a favorable trade-off rather than blanket suppression?
Extra compute vs recovery	Is the guard cheaper than constant protection at similar stability?
Trigger activations over training	Are activations meaningful or random noise?

The fair comparison test
If fixed guarding and RAFG have the same recovery but RAFG uses less compute **including monitoring**, that is useful. If it merely does more corrective work or sees extra target examples, the claim is weaker.

Go deeper — optional
You have the essential explanation above. These links are for tutorials, original evidence, or working code.
- R-TOFU benchmark — How to evaluate intermediate reasoning leakage.
- Statistical evaluation via SciPy — Confidence intervals and paired comparisons for the write-up.

SECTION 11 / RAFG RESEARCH GUIDE

When experiments do not cooperate

Pre-agreed decisions prevent wasted compute and weak claims.

1 What can fail, and what is the next sound step?

Observation	Likely meaning	Responsible next step
M1 cannot recall fictional facts	Knowledge injection was insufficient or mismatched.	Debug labels, training, templates; try a tiny subset.
M2 never forgets	Baseline unlearning is not functioning.	Inspect CE sign, alpha/beta, masking, learning rate.
M2 forgets but reasoning collapses	Excessive unlearning damage.	Tune retain term; choose a stable M2.
Nothing reappears after new training	No measurable recovery in this setting.	Report it; compare planned similar-syntax update.
Guard activates constantly	Threshold or guard is too sensitive.	Calibrate trigger on development data.
Guard beats no guard only	Could be ordinary regularization.	Compare fixed and periodic guards at matched compute.
Guard training reintroduces targets	Poor guard design or dataset overlap.	Audit prompts; monitor exact and multi-hop outputs.

2 Ablations: change one piece at a time

An ablation checks what part of the method produces any benefit. For RAFG, test direct-only monitoring versus direct+multi-hop, longer versus shorter check intervals, fixed versus M2-relative thresholds, and short versus longer correction bursts.

3 Small-model reasoning caveat

A 0.6B model may not be reliable on difficult GSM8K tasks. Use controlled one- to three-step relational reasoning as the primary new-task signal; add GSM8K as a secondary, harder check. A poor GSM8K score should not be confused with a guard failure if the original model also scores poorly.

Critical limitation
Our synthetic setting tracks a chosen adapter and cooperative authorized fine-tuning. It does not protect against someone deliberately removing the guard, nor does it guarantee certified deletion of genuine private data. Because RAFG stores target answers for future correction, it would need a different data-handling design before use for real deletion requests.

4 Predefine the go/no-go criterion

The project is worth developing further if there is measurable initial forgetting, a nontrivial recovery challenge, and RAFG improves stability–utility–cost trade-offs over a tuned baseline. If these conditions fail, report the result clearly and revise the problem rather than forcing a positive conclusion.

Go deeper — optional
You have the essential explanation above. These links are for tutorials, original evidence, or working code.
- ICML 2026: Unlearning Isn't Deletion — Why behavioral disappearance and persistent erasure differ.
- ICML 2025: Smooth robust unlearning — Competing strategies for recovery resistance.

SECTION 12 / RAFG RESEARCH GUIDE

Research contribution and reporting

A small, testable method can become a credible dissertation only with the right comparisons.

1 Our working problem statement

For project proposal
After targeted unlearning, further legitimate fine-tuning may make previously suppressed knowledge recoverable. This project tests whether a **recovery-triggered corrective objective** can stabilize forgetting in small reasoning-capable LLMs with less unnecessary intervention, while preserving new-task reasoning accuracy.

2 What we are changing vs what we reuse

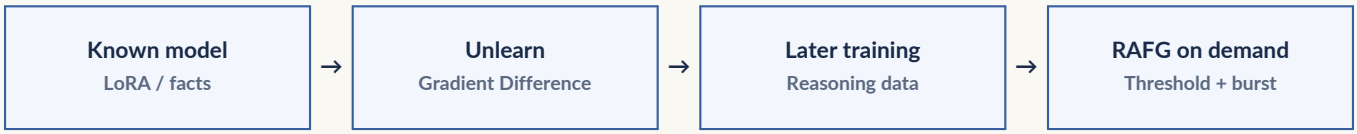
Reused, established pieces	Our proposed extra work
Qwen3 open-weight model and LoRA	One consistent adapter-based experiment pipeline.
Synthetic forget/retain split	Held-out direct and multi-hop recovery probes.
Gradient Difference unlearning	M2-calibrated monitoring and trigger.
Ordinary continued fine-tuning	Conditional burst of margin-based correction.
Fixed and periodic stabilization baselines	Matched-cost comparison with RAFG.

3 How a complete dissertation can be organized

- Introduction: machine unlearning, benign relearning, reasoning ability.
- Literature review: what prior studies establish and why this is incremental.
- Problem setup: model access, forget scope, threat model and data partitions.

- Proposed RAFG: monitoring, trigger, corrective loss and computational cost.
- Experiments: synthetic benchmark, baseline unlearning, later task training and evaluation.
- Results: recovery, utility, cost, failure cases, ablations and limitations.
- Conclusion: what improved, what did not, and what evidence supports the claim.

4 One diagram to explain to your guide



The research is **not** "we invented machine unlearning." It is "we reused a standard unlearning method and tested a small adaptive safeguard during subsequent training."

5 What to be able to explain in a viva

- Why is no direct answer not necessarily proof of forgetting?
- Why must all comparison branches start from the same M2 checkpoint?
- Why is periodic correction an essential baseline?
- What is the difference between monitoring CE and generated target leakage?
- What happens if monitoring itself costs more than RAFG saves?
- Which claims are limited to synthetic adapter-learned facts?

Most important closing principle
A successful M.Tech project does not need a groundbreaking new model. It needs a clearly defined problem, a fair small optimization, reproducible experiments, strong baselines and claims no broader than the evidence.

APPENDIX / QUICK REFERENCE

Glossary you will actually use

Keep this page beside you while reading papers or coding.

Term	Meaning in this project
LLM / reasoning model	Text predictor that can also generate intermediate solutions.
Synthetic knowledge injection	Teaching an open model fictional facts under controlled training.
LoRA	Train small added weight matrices instead of changing all base weights.
M0 / M1 / M2	Pretrained / fact-trained / unlearned checkpoints.
Forget set	Target answers and examples to suppress.
Retain set	Knowledge and tasks we aim to keep.
Cross-entropy (CE)	Surprise of a known target answer; lower CE means higher model likelihood.
Gradient Difference	Minimize retain loss while increasing forget loss.
Benign relearning	Unexpected recovery after legitimate additional fine-tuning.
RAFG trigger	Rule deciding when a short corrective update is necessary.
Guard loss	Penalty when target likelihood rises above an allowed level.
Trace leakage	Forgotten information appearing in observed generated reasoning.
Plasticity	Ability to continue learning useful new tasks.
Ablation	Remove or change one design element to identify its effect.
Matched compute	Compare methods with total computational effort accounted for.

APPENDIX / LINKED REFERENCES

Original papers, tutorials and tools

All links below are clickable. Read the short explanation first; use the original source for detailed mathematical derivations or implementation details.

Hu et al. — Unlearning or Obfuscating? (ICLR 2025) — Benign relearning motivation.
https://proceedings.iclr.cc/paper_files/paper/2025/hash/18fd48d9cbbf9a20e434c9d3db6973c5-Abstract-Conference.html

CMU ML Blog — Benign relearning, plain-English write-up — Accessible introduction.
https://blog.ml.cmu.edu/2025/05/22/unlearning-or-obfuscating-jogging-the-memory-of-unlearned-llms-via-benign-relearning/

Yoon et al. — Rethinking Benign Relearning (ICLR 2026) — Syntactic similarity and recovery.
https://proceedings.iclr.cc/paper_files/paper/2026/hash/7b1152f0646c35d9dffed261978a58cd-Abstract-Conference.html

Yoon et al. — R-TOFU (EMNLP 2025) — Reasoning-trace leakage.
https://aclanthology.org/2025.emnlp-main.265/

Fan et al. — Robust Unlearning via SAM (ICML 2025) — Existing recovery-resistance optimization.
https://proceedings.mlr.press/v267/fan25e.html

Li et al. — Update-Matched Unlearning (Oct. 2026 preprint) — New task plasticity trade-off.
https://arxiv.org/abs/2610.21230

Security-Constrained Fine-Tuning (2026 preprint/submission) — Close overlap: guarding continued fine-tuning.
https://openreview.net/pdf?id=90EZvjKMqK

Xu et al. — Unlearning Isn't Deletion (ICML 2026) — Behavioral reversibility.
https://proceedings.mlr.press/v306/xu26cd.html

Qwen3-0.6B model card — Model usage and license.
https://huggingface.co/Qwen/Qwen3-0.6B

Hugging Face PEFT — LoRA — Lightweight training.
https://huggingface.co/docs/peft/main/conceptual_guides/lora

Hugging Face TRL — SFTTrainer — Practical fine-tuning.
https://huggingface.co/docs/trl/main/en/sft_trainer

OpenUnlearning repository — Reference unlearning baselines.
https://github.com/locuslab/open-unlearning

GSM8K dataset — Free math reasoning data.
https://huggingface.co/datasets/openai/gsm8k

Kaggle efficient GPU usage — Free quota and practical constraints.
https://www.kaggle.com/docs/efficient-gpu-usage

PyTorch cross-entropy docs — Loss-function technical details.
https://docs.pytorch.org/docs/stable/generated/torch.nn.CrossEntropyLoss.html

Status: literature checked October 2026. A preprint or submission must not be described as an accepted conference paper unless publication is independently confirmed. The RAFG formulation in this guide is proposed and unvalidated.